Node.js Topics

Backend Basics

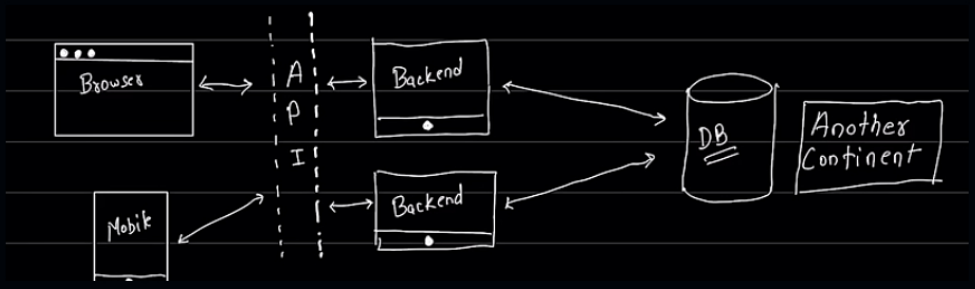
Components

1. Programming Language

- JavaScript (Node.js)
- Python (Django, Flask)
- C++ (using frameworks like CppCMS)
- PHP (Laravel, CodeIgniter)
- Java (Spring Boot)
- C# (.NET Core)

2. Database

- SQL Databases (MySQL, PostgreSQL, SQLite)
- NoSQL Databases (MongoDB, Cassandra, Redis)



- Created by Ryan Dahl in 2009
- Initially released in 2009

Alternative Runtime Environments

- `Deno`
- `Bun`

Node.js Features

- Asynchronous and Event-Driven
- Non-Blocking I/O
- Single-Threaded but Highly Scalable
- Cross-Platform
- Rich Ecosystem (npm)

Working of Node.js

- `Deno`
- `Bun`

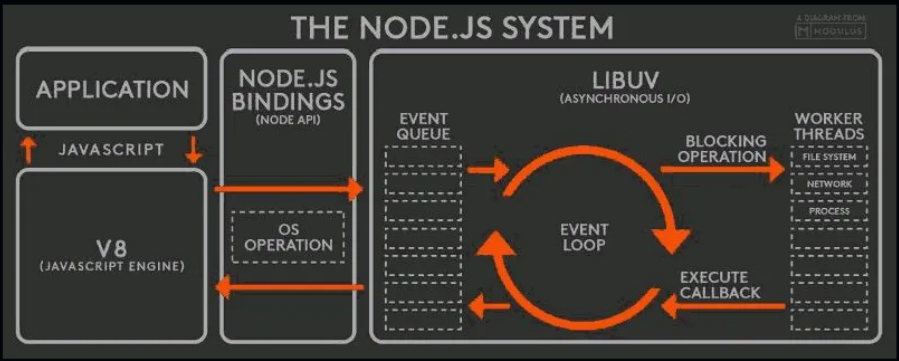
Node.js Features

- Asynchronous and Event-Driven
- Non-Blocking I/O
- Single-Threaded but Highly Scalable
- Cross-Platform

- Rich Ecosystem (npm)

Working of Node.js

- Node.js uses an event-driven architecture where operations are performed asynchronously. When a request is made, Node.js registers a callback function and continues to process other requests. Once the operation is complete, the callback function is executed.



JavaScript Essentials for Node.js

- Basic js
  - Variables, Keywords, Identifiers
  - Data Types
  - Operators (including `this` and `new` operator)
  - Expressions
  - Control Structures
    - Conditional Statements
    - Loops
  - Functions
  - Objects, Arrays , Strings
  - Date and Time , Math
  - Error Handling
  - Regular Expressions (optional)
  - DOM/BOM (optional)
- JSON
- Lexical Structure
- Classes
- Arrow Functions
- Scopes
- Template Literals (`backtick`)
- Strict Mode
- ECMAScript 2015 (ES6) and beyond
- Asynchronous JavaScript
  - Callbacks
  - Promises
  - async/await

`Hashbang` comments

A hashbang comment behaves exactly like a single line-only (`//`) comment, except that it begins with `#!` and is only valid at the absolute start of a script or module.

- Note also that no whitespace of any kind is permitted before the #!.

```
#!/usr/bin/env node
console.log("This script can be run directly from the command line!");
```

Setup Node.js Environment

- `Download` and `Install` Node.js from the official website: https://nodejs.org/
- Verify the installation by running the following commands in your terminal or command prompt:

```
node -v
npm -v
```

Dependencies vs Devdependencies

- **Dependencies** are packages that your project needs to run in production. They are listed under the "dependencies" section in package.json. Installed with `npm install <package>` or `npm install <package> --save`.
- **DevDependencies** are packages that are only needed during development (e.g., testing frameworks, build tools). They are listed under the "devDependencies" section in package.json.Installed with `npm install <package> --save-dev`.

Package Managers

- `npm` (Node Package Manager) - default package manager for Node.js
- `yarn` - alternative package manager
- `npx` - comes with npm, used to execute packages without installing them globally

```
# npm - install globally
npm install -g create-react-app
create-react-app my-app

# npx - execute without installing
npx create-react-app my-app
npx http-server
```

`package.json`: A file that contains metadata about your project and its dependencies. You can create it by running `npm init` or `yarn init`.

Semantic Versioning

Semantic versioning follows the format: `MAJOR.MINOR.PATCH`

- **MAJOR**: Breaking changes (1.0.0 → 2.0.0)
- **MINOR**: New features, backward compatible (1.0.0 → 1.1.0)
- **PATCH**: Bug fixes (1.0.0 → 1.0.1)

Version Symbols

- `^4.18.0` - Allows changes in MINOR and PATCH (≥4.18.0, <5.0.0)
- `~4.18.0` - Allows changes in PATCH only (≥4.18.0, <4.19.0)
- `4.18.0` - Exact version lock
- `*` or `x` - Any version

Basic commands

```
npm init -y          # Create a package.json file with default values
npm install <package-name>  # Install a package and add it to dependencies
npm install <package-name> --save-dev  # Install a package and add it to devDependencies
npm uninstall <package-name>  # Uninstall a package
npm update <package-name>  # Update a package to the latest version
npm list              # List installed packages
npm list --depth=0  # List installed packages without showing their dependencies
npm install          # Install all dependencies listed in package.json
npm install <package-name>@<version>  # Install a specific version of a package
npm run <script-name>  # Run a script defined in package.json under "scripts"
npm search <pkg>                        # Search npm registry
npm outdated                            # Check for outdated packages
npm audit                               # Check for security vulnerabilities
npm audit fix                           # Auto-fix vulnerabilities
```

`Shortcuts` of above commands

1. install = i
2. uninstall = un
3. update = up
4. list = ls
5. save-dev = D
6. save = S
7. global = g
8. depth = d

Node.js vs Browser

Node.js and browsers are both environments for executing JavaScript, but they have different use cases and capabilities.

Key Differences

Feature	Node.js	Browser

Purpose	Server-side applications	Client-side applications
Environment	Runs on the server	Runs on the user's device
Execution Context	Node.js runs on the server	while browsers run on the client (user's device).
APIs	Node.js provides APIs for file system access, networking, and other server-side tasks.	Browsers have APIs for DOM manipulation, user interaction, and web-specific features.
Global Objects	In Node.js, the global object is `global`	while in browsers, it is `window`.
Modules	Node.js uses `CommonJS` or `ES modules`	while browsers primarily use `ES modules`.

Development vs Production

- Development mode is used during the development phase of an application, while production mode is used when the application is deployed and running in a live environment.
- In development mode, features like detailed error messages, debugging tools, and hot-reloading are often enabled to facilitate the development process. In production mode, these features are typically disabled to improve performance and security.

Backend `File Structure`

- `public`
- `src`
  - `controllers`
  - `models`
  - `routes`
  - `db`
  - `middlewares`
  - `utils`
  - app.js (main application logic)
  - index.js
  - constants.js
- .env
- .gitignore
- package.json
- package-lock.json

Server Basics

What is a Server?

A server is a computer program or a machine that waits for requests from clients and responds to them. In the context of web development, a server hosts web applications and serves content to users over the internet.

How Does a Server Work?

1. **Listening for Requests**: The server listens for incoming requests from clients (e.g., web browsers).
2. **Processing Requests**: When a request is received, the server processes it, which may involve querying a database, performing calculations, or accessing files.
3. **Sending Responses**: After processing the request, the server sends a response back to the client, which may include HTML, JSON, or other data.

Common Server Technologies

- **Node.js**: A JavaScript runtime built on Chrome's V8 engine, commonly used for building scalable network applications.
- **Express.js**: A minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications.

```
// Nodejs Server code example
const http = require('http');
const hostname = '127.0.0.1'; // localhost
const port = 3000;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World\n');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
```

```
});
```

Nodejs `response` and `request` Object Methods

Method	Description	Example
`res.writeHead(statusCode, headers)`	Sets the HTTP status code and headers for the response.	`res.writeHead(200, {'Content-Type': 'text/plain'});`
`res.end([data])`	Ends the response process. Optionally, you can send data as the response body.	`res.end('Hello World\n');`
`res.setHeader(name, value)`	Sets a specific header for the response.	`res.setHeader('Content-Type', 'application/json');`
`res.getHeader(name)`	Retrieves the value of a specific header from the response.	`const contentType = res.getHeader('Content-Type');`
`res.statusCode`	Gets or sets the HTTP status code for the response.	`res.statusCode = 404;`
`res.statusMessage`	Gets or sets the HTTP status message for the response.	`res.statusMessage = 'Not Found';`
`res.finished`	Indicates whether the response has been sent to the client.	`console.log(res.finished);`
`res.writable`	Indicates whether the response is writable.	`console.log(res.writable);`
`res.readable`	Indicates whether the response is readable.	`console.log(res.readable);`
`res.write(data)`	Writes data to the response body.	`res.write('Hello '); res.write('World!');`
`req.url`	Returns the URL of the request.	`console.log(req.url);`
`req.method`	Returns the HTTP method of the request (e.g., GET, POST).	`console.log(req.method);`
`req.headers`	Returns the headers of the request as an object.	`console.log(req.headers);`
`req.body`	Returns the body of the request (for POST requests).	`console.log(req.body);`
`req.params`	Returns the route parameters of the request.	`console.log(req.params);`
`req.query`	Returns the query string parameters of the request.	`console.log(req.query);`
`req.cookies`	Returns the cookies sent by the client.	`console.log(req.cookies);`

Fundamentals

commonjs vs esm

- CommonJS (CJS) and ECMAScript Modules (ESM) are two different module systems used in JavaScript for organizing and reusing code.
- CommonJS is primarily used in Node.js, while ESM is the standard module system for JavaScript and is supported in both browsers and Node.js (from version 12 onwards).

`import` statement

for use in files , include below `key-value`

```
// package.json
{
   ... ,
  "type": "module",
   ...
}
```

Default Behavior in Node.js

`cjs`

```
// CommonJS follow sync way

// CommonJS (require) is the default module system in Node.js.
```

`esm`

```
// ES Module js follow async way

// ES Modules (import) can be enabled by: Using .mjs file extension.
// Setting "type": "module" in package.json
```

`require` and `import` together

```
// Using Both require and import in the Same File


//  Using require inside an ES Module (import enabled)
// By default, require is not available in ES modules. To use require, you need to import it dynamically:


import { createRequire } from 'module';
const require = createRequire(import.meta.url);

const fs = require('fs'); // Now we can use require
console.log(fs.readFileSync('file.txt', 'utf8'));
```

path

- In production path management is very important to avoid errors related to file locations.
- Use `path` module to handle file paths.

example:

```
import path from 'path';

// Joining paths
const fullPath = path.join(__dirname, 'file.txt');
console.log(fullPath);

// Normalizing paths
const normalizedPath = path.normalize('/foo/bar//baz/asdf/quux/..');
console.log(normalizedPath);
```

Usage of `__dirname` in Node.js

```
// Getting the current directory path:
console.log(__dirname);
// This will print the absolute path of the directory where the script is located.
```

- Limitations

```
// __dirname Not available in ES modules. Instead, use:

import { dirname } from 'path';
import { fileURLToPath } from 'url';

const __filename = fileURLToPath(import.meta.url);
const __dirname = dirname(__filename);

//or

// const __dirname = import.meta.dirname
```

package.json

- `package.json` is a file that contains metadata about your Node.js project and its dependencies. It is used by npm (Node Package Manager) to manage the project's packages and scripts.

Key Fields in package.json

- `name` : The name of the project.
- `version` : The version of the project.
- `description` : A brief description of the project.
- `main` : The entry point of the application (e.g., index.js).
- `scripts` : A set of commands that can be run using npm (e.g., start, test).
- `dependencies` : A list of packages required for the project to run.
- `devDependencies` : A list of packages required for development (e.g., testing frameworks).
- `type` : Specifies the module system (e.g., "commonjs" or "module").

example:

```
{
  "name": "my-nodejs-project",
  "version": "1.0.0",
  "description": "A sample Node.js project",
  "main": "index.js",
```

```
  "scripts": {
    "start": "node index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "dependencies": {
    "express": "^4.17.1"
  },
  "devDependencies": {
    "nodemon": "^2.0.7"
  },
  "type": "module"
}
```

Dotenv

DotEnv setup as module in `package.json` file

If changing in `.env` file need to restart server

```
// for older nodejs version to access .env file variables in any file with `process.env.*`
"dev": "nodemon -r dotenv/config --experimental-json-modules src/index.js",
```

use this code

```
// require('dotenv').config({path:'./env'})

import dotenv from 'dotenv'

dotenv.config({path:'./env'})

console.log(process.env.DB_USERNAME);
```

```
// .env file
DB_USERNAME=myusername
DB_PASSWORD=mypassword
```

`Express` .js

Express.js is a web application framework for Node.js designed for building web applications and APIs. It simplifies the process of handling requests, routing, and middleware integration.

- Express provide `err,req,res,next` parameters on server in each function

Key Features

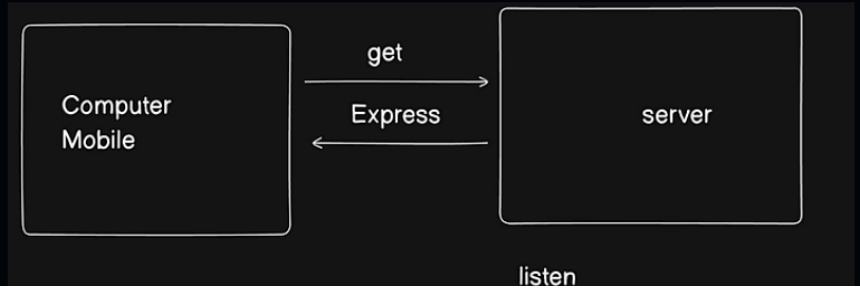
- **Middleware Support**: Easily add middleware functions to handle requests and responses.
- **Routing**: Define routes for different HTTP methods and URLs.
- **Request and Response Objects**: Simplified request and response objects for easier handling.
- **Error Handling**: Centralized error handling for your application.

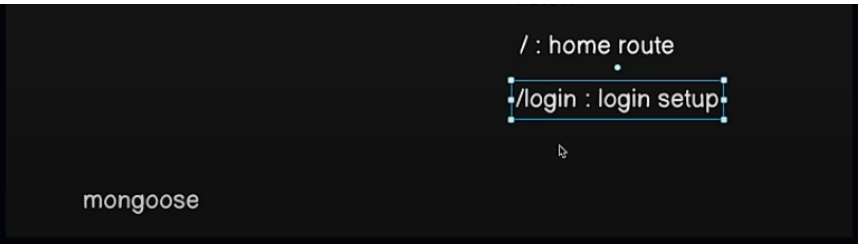
Basic Example

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello World!');
});

app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}/`);
});
```





Expressjs `Request` and `Response` Object Methods

Method	Description	Example
`res.send([body])`	Sends a response to the client. The body can be a string, object, or buffer.	`res.send('Hello World!');`
`res.json([body])`	Sends a JSON response to the client.	`res.json({ message: 'Hello World!' });`
`res.status(code)`	Sets the HTTP status code for the response.	`res.status(404).send('Not Found');`
`res.redirect([status], url)`	Redirects the client to a different URL.	`res.redirect('/new-url');`
`res.render(view, [locals], callback)`	Renders a view template and sends the HTML response.	`res.render('index', { title: 'Home' });`
`req.params`	Returns the route parameters of the request.	`console.log(req.params.id);`
`req.query`	Returns the query string parameters of the request.	`console.log(req.query.page);`
`req.body`	Returns the body of the request (for POST requests).	`console.log(req.body);`
`req.get(field)`	Returns the value of a specific header field from the request.	`console.log(req.get('Content-Type'));`
`req.cookies`	Returns the cookies sent by the client.	`console.log(req.cookies);`
`req.path`	Returns the path part of the request URL.	`console.log(req.path);`
`req.method`	Returns the HTTP method of the request (e.g., GET, POST).	`console.log(req.method);`
`req.url`	Returns the URL of the request.	`console.log(req.url);`

File Uploads and Static Files

Serve static files (CSS, images, JS) using Express:

```
app.use(express.static('public'));
// Access at: http://localhost:3000/style.css
```

Error Handling

Centralized error handling middleware (must have 4 parameters).

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500).json({
    message: err.message || 'Internal Server Error'
  });
});
```

File Handling

Reading and Writing Files

```
import fs from 'fs';

// Synchronous (blocking)
const content = fs.readFileSync('file.txt', 'utf8');

// Asynchronous with callback
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) console.error(err);
  console.log(data);
});

// Asynchronous with promises
```

```
fs.promises.readFile('file.txt', 'utf8')
  .then(data => console.log(data))
  .catch(err => console.error(err));

// Write file
fs.writeFileSync('output.txt', 'Hello World');

fs.writeFile('output.txt', 'Hello World', (err) => {
  if (err) console.error(err);
});
```

Databases

SQL vs NoSQL

Feature	SQL Databases	NoSQL Databases
Data Storage	Structured tables with rows and columns	Flexible document, key-value, graph, or columnar storage
Schema	Fixed schema with strict rules	Dynamic schema (schema-less)
Scalability	Vertical scaling (adding more power to one server)	Horizontal scaling (adding more servers)
ACID Compliance	Fully ACID compliant	Often BASE compliant
Query Language	Structured Query Language (SQL)	Varies by type (e.g., MongoDB's aggregation pipeline)
Use Cases	Complex queries, transactions	Real-time analytics, content management, large-scale applications

SQL Databases

- MySQL

```
//connection example
const mysql = require('mysql');
const connection = mysql.createConnection({
  host: 'localhost',
  user: 'yourusername',
  password: 'yourpassword',
  database: 'yourdatabase' // no need to create db first it will create automatically and it optional
});
connection.connect((err) => {
  if (err) throw err;
  console.log('Connected to MySQL database!');
});

// executing queries
connection.query('SELECT * FROM users', (err, results) => {
  if (err) throw err;
  console.log(results);
});

// dynamic query
const userId = 1;
connection.query('SELECT * FROM users WHERE id = ?', [userId], (err, results) => {
  if (err) throw err;
  console.log(results);
});

// close connection
connection.end();
```